LAB 4 - VIBE CODING
AI-ASSISTED SESSION LOG

Whack-a-Mole | Assigned repository: SETAPESU26/48_whack-a-mole

Objective and source requirements

The Lab 4 handout asks the student to clone the assigned Python game, record 10 seconds before changes, use an LLM to fix the broken code and add every feature listed in the assigned README, record 10 seconds after changes, push the updated code to a personal repository, and keep separate commits for each task. The assigned README lists four tasks: refine collision detection, add a Game Over screen, add replay with Easy/Medium/Hard difficulty, and add sound feedback for hit, miss, and round end.

Conversation and implementation record

User request

Upload before/after videos, updated code, and Chat history as a document/PDF under Lab-4 in the personal lab repository; use the classmate Lab-4 folder as an example.

Assistant review

Identified the assigned repository from the supplied screenshot and read the Lab 4 handout, the assigned starter README, and the starter Python files. The starter README specifies the four required features and the video/chat-history requirements.

Bug diagnosis

In game/game_engine.py, the original click handler iterated over every hole and awarded a point for every active hole whose rectangle contained the click. In game/hole.py, each hitbox was 150 pixels wide, causing neighboring hitboxes to overlap.

Task 1 - collision detection

Reduced the hole hitbox based on grid spacing and added nearest-active-mole selection so a single click can score at most one point. A miss increments the miss count and does not change the score.

Task 2 - Game Over

Added an in-window Game Over display with final score and best score when the countdown reaches zero. Added a one-time round-end sound trigger.

Task 3 - replay and difficulty

Added Easy, Medium, and Hard options. The selected difficulty changes spawn probability and the duration a mole remains visible. Selecting a mode resets score, timer, and active holes for a new round.

Task 4 - sound feedback

Added short generated tones for successful hit, missed click, and round end. Audio initialization is guarded so the game can still run without an available audio device.

Validation

Ran five unit tests with a lightweight Pygame stub: non-overlapping hitboxes, at-most-one score for one click, miss handling, timer-triggered Game Over, and replay reset/difficulty selection. All five tests passed. Pygame was not installed in the execution environment, so live graphics and sound were not run here.

Evidence still required

The before.mp4 and after.mp4 clips must be captured from actual gameplay on the student Mac. No video was fabricated. The PDF is a session log/summary and is not a full native ChatGPT export or share link.

Updated files

- main.py
- game/game_engine.py
- game/hole.py
- requirements.txt
- tests/test_game_engine.py
- README.md
- VIDEO_CAPTURE_GUIDE.md

Repository boundary: the assigned organization repository is read-only and was not modified. Updated code is intended for the student’s own personal lab repository. This log reflects the work and checks performed in this session; it does not claim that gameplay recordings or a GitHub upload have already been completed.